

Dylan Trenck, Matthew Bosworth, Christopher Lu, Mohamed Abdou, Suphakrit Jinaongkan,
Zarah Maullon

April 14, 2026

COP 4610 – Operating Systems

Professor Sanjay Ahuja

Paper Review: Analysis of Library Dependency Networks of Package Managers Used in iOS Development

The paper “Analysis of Library Dependency Networks of Package Managers Used in iOS Development” was written by Kristiina Rahkema, Dietmar Pfahl, and Rudolf Ramler. It was published in 2023 at the IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft). The paper presents the first analysis of how library dependency networks have changed over time in the Swift ecosystem. The Swift ecosystem includes three package managers used in iOS and macOS development: CocoaPods, Carthage, and Swift Package Manager. The authors built a dataset of over 60,000 libraries and 572,000 library versions and used this data to study how the number of libraries, versions, and dependencies has grown.

The study is organized around three research questions. The first question asks how the combined library dependency network of the Swift ecosystem has evolved over time. The second question looks at how the networks of each individual package manager have grown. The third question examines how the number of direct and transitive dependencies has changed. To answer these questions, the authors collected dependency data from the configuration files used by each package manager, such as Podfiles for CocoaPods, Cartfiles for Carthage, and Package.swift files for Swift PM. They used git commit timestamps to date each library version and then plotted cumulative growth curves and monthly dependency snapshots.

The results show several important findings. First, while the number of libraries in the Swift ecosystem grew rapidly after the release of the Swift programming language in 2014, that growth has slowed down since 2019. This is different from what previous studies found for other ecosystems like npm and Maven, where growth was linear or exponential. The authors believe this slowdown is because developers are moving to Swift Package Manager, which became much easier to use after Apple added native Xcode support in 2019. CocoaPods, the oldest and largest package manager with about 88,000 libraries, is seeing slower growth. Carthage has almost stopped growing entirely. Swift PM, despite being the newest, is growing the fastest.

Another key finding is about the number of dependencies. The mean number of direct dependencies is around two to three for all three package managers, with a slight upward trend. The total number of dependencies, including transitive ones, grew from about three to five and a half between 2016 and 2022. When compared to other software ecosystems, the Swift ecosystem has far fewer transitive dependencies. For example, the median number of transitive dependencies for libraries in the Cargo, NuGet, and npm ecosystems is 41, 27, and 21, respectively. In the Swift ecosystem, the median is only two. This is an important finding because fewer transitive dependencies means a smaller attack surface for security vulnerabilities. The authors connect this to the well-known Log4J incident, where a vulnerability in a single logging library affected a large portion of the software industry because of deep transitive dependency chains.

This paper is relevant to operating systems concepts in several ways. Package managers operate as part of the software build pipeline that interfaces directly with the operating system. The process of resolving and linking library dependencies is similar to how an operating system manages shared libraries and dynamic linking at runtime. When iOS loads an application, the kernel and the dynamic linker must resolve dependencies between frameworks and shared libraries, much like the dependency chains studied in this paper. Additionally, the security implications of transitive dependencies connect to the iOS sandboxing model. Each iOS app runs in its own sandbox with strict access controls enforced by the XNU kernel, and having fewer

dependency chains reduces the risk that a vulnerability in one library could be exploited to escape the sandbox or compromise user data.

Overall, we found this paper to be a well-structured and informative study. We agree with the authors' main finding that the Swift ecosystem is consolidating around Swift Package Manager, and we think this is a positive development because having a single official tool reduces confusion for developers. We also found the cross-ecosystem comparison very useful. The fact that the Swift ecosystem has a median of only two transitive dependencies compared to 21 to 41 in other ecosystems suggests that Apple's tightly controlled platform and its emphasis on self-contained frameworks contribute to leaner dependency trees. One limitation we noticed is that the paper does not analyze whether specific vulnerable library versions are present in the dependency chains, which would have strengthened the security argument. Despite this, the paper fills an important gap in the research by being the first to study the Swift ecosystem's dependency networks, and the findings are useful for both iOS developers and researchers interested in software supply chain security.